

MISP Foundations for Integration

MISP Integration Workshop

The data model: events, attributes, objects

Tags, taxonomies & galaxies

Feeds & servers

PyMISP & the REST API

Filtering exports

MISP modules

The data model

Events, attributes, objects — what every integration consumes.

- **Event** — a container for a report/incident; the unit of sharing and publishing.
- **Attribute** — a single data point (IP, domain, hash, URL) with a **type** and **category**.
- **Object** — a structured group of attributes following a template (e.g. `file`, `network-connection`).

Integrations read attributes; events give them context and scope.

Tags, taxonomies & galaxies

Machine-readable labels that drive filtering and automation downstream.

- **Tags** — free-form or taxonomy-backed labels attached to events/attributes.
- **Taxonomies** — namespaced, agreed vocabularies — e.g. `tlp:clear`, `tlp:amber`, confidence, workflow state.
- **Galaxy clusters** — rich knowledge objects — e.g. `misp-galaxy:mitre-attack-pattern`.

Downstream tools filter on these tags — consistent tagging = reliable automation.

Feeds & servers

Ingesting external intelligence.

- **Feeds** — pull indicators from remote sources (MISP feeds, CSV, free-text) on a schedule.
- **Servers** — synchronise events with other MISP instances (push / pull), respecting distribution and TLP.
- Both populate your instance so integrations have data to act on.

Curate what you ingest — feeds are the top of the quality funnel.

PyMISP & the REST API

The backbone of every integration.

- **REST API** — everything the UI does is an authenticated HTTP call (`/events/restSearch`, `/attributes/restSearch`).
- **PyMISP** — the official Python client wrapping that API.
- **Auth keys** — per-user API keys scope access; treat them as secrets.

If a tool integrates with MISP, it speaks REST — usually via PyMISP.

Filtering exports

Push only what detection tools should act on.

- **to_ids flag** — export only attributes marked for detection.
- **Tags** — include/exclude by taxonomy (e.g. only `tlp:clear` / `tlp:green`).
- **Attribute types** — restrict to what the sensor understands (IPs, domains, hashes).

Filtering at export keeps noise and TLP-restricted data out of your sensors.

MISP modules

Enrich IOCs and interact with external services.

- **Expansion modules** — enrich an attribute (passive DNS, WHOIS, VirusTotal, Shodan) on demand.
- **Import / export modules** — parse external formats in, push formats out.
- Run as a separate `misp-modules` service the instance calls.

Enrichment adds context without leaving the MISP workflow.

Hands-on

Hands-on: end-to-end

Take one IOC through the full integration path.

1. **Create** an event (set distribution + TLP).
2. **Tag** it (`tlp:clear` , a galaxy cluster).
3. **Add** attributes and flag IOCs `to_ids=true` .
4. **Enrich** an attribute with a MISP module.
5. **Publish** the event.
6. **Pull** it via PyMISP (`restSearch`) and inspect the result.

This is exactly what a downstream integration does — you're doing it by hand first.

Questions?